certora

Security Assessment

Eigenlayer Incentive Council

January 2026

Prepared for Eigenlayer

Table of Contents

Project Summary 3
- Project Scope 3
- Project Overview 3
- Protocol Overview 4

Assessment Methodology 5

Threat and Security Overview 6
- Assets 7
- Actors 7
- Trust Assumptions 8
- Attack Vectors 9
- Findings Summary 11
- Severity Matrix 11

Detailed Findings 12
- **High Severity Issues** 14
  - H-01: createEigenDARewardsSubmission incorrectly pulls funds from AVS instead of EmissionsController 14
- **Medium Severity Issues** 16
  - M-01: Hardcoded bEIGEN address in EmissionsController hinders protocol resilience to forks 16
  - M-02: Loss of protocol fee when distributing rewards via EmissionsController due to incorrect opt-in check 17
- **Low Severity Issues** 19
  - L-01: Inconsistent reward amount validation against MAX_REWARDS_AMOUNT in RewardsCoordinator 19
  - L-02: RewardsCoordinator.createAVSRewardsSubmission is vulnerable to event spamming and off-chain griefing 20
- **Informational Severity Issues** 21
  - I-01: Use standard initializer for safer ownership assignment 21
  - I-02: Outdated documentation on pausable flags in RewardsCoordinatorStorage 22
  - I-03: EmissionsController._checkDistribution fails to validate operator sets for EigenDA distributions 23
  - I-04: Outdated documentation for createRewardsForAllEarners in IRewardsCoordinator 24

Disclaimer 25

About Certora 25

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Eigenlayer Incentive Council	https://github.com/Layr-Labs/eigenlayer-contracts/tree/release-dev/incentive-council	4287d2f (start commit) bf90f45 (final commit)	Solidity

Project Overview

This document describes the security review of **Eigenlayer Incentive Council**. The work was undertaken from **January 20th, 2026** to **January 30th, 2026**.

The following contract list is included in our scope:

- src/contracts/core/EmissionsController.sol
- src/contracts/core/storage/EmissionsControllerStorage.sol
- src/contracts/core/RewardsCoordinator.sol
- src/contracts/core/storage/RewardsCoordinatorStorage.sol

Pull requests:

PR-1681
PR-1691
PR-1703
PR-1705

The team performed a manual audit of all the files in scope**.** During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

The scope involves implementing ELIP-012's Incentive Council framework for EigenLayer, which transitions from rigid programmatic EIGEN emissions to a flexible, governance-controlled rewards system. The EmissionsController contract consolidates token minting and distribution using configurable gauge weights, allowing the Incentive Council to dynamically allocate emissions across different distribution types (protocol-wide rewards, operator set stake-based rewards, manual distributions, and EigenDA rewards) without requiring contract upgrades. A key objective is directing rewards toward productive stake—capital backing active AVSs with slashable or redistributable commitments—while reducing emissions to idle stake.

The implementation introduces an opt-in 20% protocol fee on AVS reward submissions through RewardsCoordinator, enabling value capture that can be used for token buybacks. The system operates on an epoch-based model with permissionless execution via the pressButton() function, while maintaining security through protections against reentrancy and out-of-gas attacks. This architectural change empowers the Incentive Council to respond to ecosystem needs with agility, aligning token economics with genuine network activity and fee generation rather than static allocation formulas.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

Security Overview

The Incentive Council's safety depends on correct epoch state transitions, the integrity of the EmissionsController's gauge-weighted distribution logic, and proper interaction with the RewardsCoordinator across multiple reward submission types. We focused on invariants around epoch-based minting (EIGEN minted at most once per epoch), weight constraints (totalWeight ≤ 10,000 bips), distribution lifecycle (distributions added in epoch N excluded from processing until N+1), emission caps (distributed amounts ≤ EMISSIONS_INFLATION_RATE per epoch), and fee opt-in mechanisms. Attack vectors considered include reentrancy exploitation during RewardsCoordinator calls, out-of-gas griefing to block distribution processing.

Manual review covered EmissionsController.sol and its interactions with RewardsCoordinator across the distribution processing pipeline (pressButton, sweep, fee handling, EigenDA special case), with particular attention to the low-level call pattern used for reentrancy/OOG protection and the epoch state machine.

Design Overview

The EmissionsController consolidates EIGEN minting and distribution into a single contract using configurable gauge weights, replacing the rigid TokenHopper→ActionGenerator pipeline with flexible, governance-controlled allocation. The design introduces six distribution types (RewardsForAllEarners, OperatorSetTotalStake, OperatorSetUniqueStake, EigenDA, Manual, Disabled) that share a unified processing loop but require different validation and routing logic, which creates configuration complexity—particularly around OperatorSet permissions and strategy ordering. Because the contract uses low-level calls to RewardsCoordinator to prevent individual distribution failures from blocking the entire epoch, failed distributions are silently skipped (except for reentrancy/OOG), leaving undistributed EIGEN in the contract until sweep() is called.

The epoch state machine strictly enforces that distributions can only be added/updated when isButtonPressable() is false, preventing mid-epoch weight changes, but this creates operational constraints where the Incentive Council must wait for full epoch processing before making

adjustments. The dual opt-in fee mechanism in RewardsCoordinator (submitter opts in, Council sets recipient) adds revenue capture without breaking existing integrations, but relies on correct configuration and the assumption that the EmissionsController itself opts into fees when appropriate. Lastly, the design appropriately isolates concerns (minting in EmissionsController, distribution in RewardsCoordinator), but could benefit from stronger on-chain guardrails against stale distributions consuming gas, minimum weight thresholds, or automatic weight cleanup when distributions end, rather than relying entirely on off-chain monitoring and Incentive Council operational discipline.

Threat Model

Assets

EIGEN and bEIGEN Tokens: Protocol emission tokens controlled by EmissionsController. EmissionsController mints bEIGEN and wraps it to EIGEN during pressButton() execution, representing the incentive budget allocated across distribution types.

Protocol Fee Revenue: 20% fee on AVS reward submissions accumulating in feeRecipient for token buybacks or value accrual.

Staker and Operator Rewards: EIGEN emissions distributed via RewardsCoordinator based on stake participation in AVSs.

Emission Budget Authority: Gauge weights controlled by Incentive Council determining allocation of EMISSIONS_INFLATION_RATE across distribution types.

Epoch State Integrity: Monotonic epoch progression ensuring controlled inflation via once-per-epoch minting.

Actors

Protocol Council: Governance authority setting Incentive Council address and approving ELIP-012 implementation.

Incentive Council: Multisig controlling distribution configuration and gauge weights without requiring contract upgrades.

EmissionsController Contract: Autonomous minter routing EIGEN to RewardsCoordinator based on configured distributions.

AVS Contracts: Services registering OperatorSets and potentially opting into protocol fees.

Operators and Stakers: End recipients of EIGEN emissions through RewardsCoordinator.

Permissionless Button Pressers: Any address calling pressButton() and sweep(), enabling liveness but also potential griefing.

Off-chain Indexers: Services processing RewardsCoordinator events for Merkle root calculations, vulnerable to spam.

Trust Assumptions

Incentive Council Operational Discipline: Proper validation of distribution configurations including OperatorSet references, strategy ordering, weight allocations, and timely stale distribution removal.

RewardsCoordinator Correctness: Proper processing and routing of tokens through createRewardsForAllEarners, createTotalStakeRewardsSubmission, createUniqueStakeRewardsSubmission, and createEigenDARewardsSubmission.

bEIGEN and EIGEN Token Stability: Immutable bEIGEN address assumes no upgrades, migrations, or forks; wrapping mechanism and mint() function must remain stable.

Epoch Liveness from External Callers: Honest actors calling pressButton() each epoch; skipped epochs result in permanent emission loss.

AllocationManager OperatorSet Integrity: Correct operator registration validation and stake accounting.

Low-Level Call Safety: Silent failures acceptable for non-critical errors; reentrancy and OOG detection mechanisms sufficient.

Fee Opt-in Model: AVSs and EmissionsController correctly opt-in to protocol fees when appropriate.

Attack Vectors

Token Source Confusion in EigenDA Distributions: createEigenDARewardsSubmission pulls tokens from avs parameter instead of msg.sender (EmissionsController), causing distribution failures or unintended AVS billing. *[H-01]*

Protocol Fee Bypass Through Caller Identity: Fee checks use msg.sender (EmissionsController) instead of AVS address in createUniqueStakeRewardsSubmission/createTotalStakeRewardsSubmission, losing 20% fee revenue when AVS is opted-in but EmissionsController is not. *[M-02]*

Hardcoded bEIGEN Address Lock-in: Immutable BACKING_EIGEN prevents adaptation to forks or upgrades; proxy upgrade fails if new bEIGEN lacks mint() function. *[M-01]*

Reentrancy and Out-of-Gas Griefing: Protections detect OpenZeppelin reentrancy signatures and empty return data (OOG), but edge cases around other revert patterns could allow silent failures.

Inconsistent MAX_REWARDS_AMOUNT Validation: Cap applied pre-fee in standard submissions but post-fee in operator-directed submissions, allowing bypass by up to 20%. *[L-01]*

Event Spam and Indexer DoS: Permissionless createAVSRewardsSubmission allows worthless token submissions, polluting storage and griefing off-chain indexers. *[L-02]*

Malformed Strategy Arrays: Incorrect strategy ordering or non-whitelisted tokens cause silent failures; extremely large arrays trigger OOG protection blocking epochs.

Missing Epoch Button Presses: Each missed epoch permanently forfeits EMISSIONS_INFLATION_RATE; no accumulation or catch-up mechanism exists.

Zero Address Initialization Risk: Direct _transferOwnership call without validation could set owner to address(0), making contract unowned. *[I-01]*

EigenDA OperatorSet Validation Gap: _checkDistribution omits EigenDA type from OperatorSet registration validation despite using distribution.operatorSet.avs. *[I-03]*

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	1	1	1
Medium	2	2	1
Low	2	2	1
Informational	4	4	2
Total	9	9	5

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
		Likelihood		

Detailed Findings

ID	Title	Severity	Status
H-01	createEigenDARewardsSubmission incorrectly pulls funds from AVS instead of EmissionsController	High	Fixed
M-01	Hardcoded bEIGEN address in EmissionsController hinders protocol resilience to forks	Medium	Acknowledged
M-02	Loss of protocol fee when distributing rewards via EmissionsController due to incorrect opt-in check	Medium	Fixed
L-01	Inconsistent reward amount validation against MAX_REWARDS_AMOUNT in RewardsCoordinator	Low	Fixed
L-02	RewardsCoordinator.createAVSRewardsSubmission is vulnerable to event spamming and off-chain griefing	Low	Acknowledged
I-01	Use standard initializer for safer ownership assignment	Informational	Acknowledged
I-02	Outdated documentation on pausable flags in RewardsCoordinatorStorage	Informational	Fixed

I-03	EmissionsController._checkDistribution fails to validate operator sets for EigenDA distributions	Informational	Acknowledged
I-04	Outdated documentation for createRewardsForAllEarners in IRewardsCoordinator	Informational	Fixed

High Severity Issues

H-01: createEigenDARewardsSubmission incorrectly pulls funds from AVS instead of EmissionsController

Severity: High	Impact: High	Likelihood: Medium
Files: src/contracts/core/RewardsCoordinator.sol#L91-L97	Status: Fixed	

Description:

The EmissionsController is designed to handle rewards. In the pressButton function, new EIGEN tokens are minted, residing in the EmissionsController contract itself.

When processing a distribution of type EigenDA, the controller calls RewardsCoordinator.createEigenDARewardsSubmission, passing the AVS contract address as the avs argument.

However, the implementation of createEigenDARewardsSubmission in the RewardsCoordinator delegates logic to _createAVSRewardsSubmission, which uses the avs argument as the source of funds:

```
    function _createAVSRewardsSubmission(
    address avs,
    RewardsSubmission[] calldata rewardsSubmissions
  ) internal {
    for (uint256 i = 0; i < rewardsSubmissions.length; i++) {
        // ... validation ...

        // This line attempts to pull tokens from 'avs'
        // But the tokens are held by 'msg.sender' (EmissionsController)
        rewardsSubmission.token.safeTransferFrom(avs, address(this), rewardsSubmission.amount);

        // ... fee logic and event emission ...
    }
```

```
}
```

This logic creates a fundamental mismatch in the flow of funds. The EmissionsController holds the minted tokens intended for the reward, but the RewardsCoordinator attempts to bill the AVS address directly.

As a result 2 possible scenarios exist:

1. If the AVS hasn't approved RewardsCoordinator to spend the token amount or if the AVS doesn't have that much balance of the Eigen token the call to createEigenDARewardsSubmission will fail and that distribution will be skipped. The incentive council will have to distribute those rewards manually.

2. The bigger problem is when the AVS has given approval to the RewardsCoordinator and has enough balance of Eigen tokens, in which case the AVS will get billed for this reward that it didn't expect while the EmissionsController has already minted those rewards and are just sitting in the contract.

Recommendations:

Modify _createAVSRewardsSubmission to pull the tokens from msg.sender instead of the avs argument.

Customer Response:

Fixed in commit 355160b0

Fix Review:

Fixed

Medium Severity Issues

M-01: Hardcoded bEIGEN address in EmissionsController hinders protocol resilience to forks

Severity: Medium	Impact: High	Likelihood: Low
Files: src/contracts/core/EmissionsController.sol#L100	Status: Acknowledged	

Description:

In EmissionsController, the addresses for EIGEN and BACKING_EIGEN (bEIGEN) are defined as immutable variables set during the constructor. While this provides gas efficiency, it creates a rigid dependency on specific token contract instances. In the event of a protocol fork or a migration of the underlying bEIGEN implementation (e.g., from bEIGEN_v1 to bEIGEN_v2), the EmissionsController would be unable to interact with the new versions without a full contract upgrade or redeployment. This is particularly relevant as EIGEN itself wraps bEIGEN, and the EmissionsController performs minting and wrapping operations using these hardcoded addresses.

Recommendations:

Ensure the upgrade path via the proxy is well-documented, or consider making these addresses modifiable by the owner in case of forks. Also consider the lack of mint() function in the forked bEIGEN token, which would break the contract even after an upgrade

Customer Response:

Acknowledged

"The BACKING_EIGEN address is intentionally immutable as part of the EmissionsController's core design. This decision was made for the following reasons:

- *Simplicity and Gas Efficiency: Immutable storage provides significant gas savings and removes the complexity of governance-controlled token address updates.*
- *Proxy Upgrade Path: The EmissionsController is deployed behind an upgradeable proxy. In the unlikely event of a bEIGEN migration or fork, we can deploy a new implementation pointing to the new token contract and upgrade via the proxy pattern.*
- *Controlled Upgrade Surface: Making token addresses mutable increases the governance attack surface. An immutable design reduces the risk of governance compromise leading to unauthorized token substitutions.*
- *Low Likelihood: The bEIGEN and EIGEN tokens are foundational protocol primitives with no planned migrations. The scenario described (fork requiring new bEIGEN) is considered extremely low probability.*

Mitigation Strategy:

- *Comprehensive upgrade documentation exists for proxy upgrades in the event of token migrations*
- *Multi-sig and timelock controls on proxy upgrades provide security guarantees*
- *Emergency pause functionality allows halting emissions if issues arise"*

Fix Review:

Acknowledged

M-02: Loss of protocol fee when distributing rewards via EmissionsController due to incorrect opt-in check

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: src/contracts/core/RewardsCoordinator.sol#L245	Status: Fixed	

Description:

The RewardsCoordinator allows reward submitters (typically AVSs) to opt-in to paying protocol fees via setOptInForProtocolFee. When rewards are submitted, the _takeProtocolFee function checks isOptedInForProtocolFee mapping to determine if a percentage of the rewards should be diverted to the feeRecipient.

In the functions createUniqueStakeRewardsSubmission and createTotalStakeRewardsSubmission, the contract determines fee eligibility by checking msg.sender:

```
// RewardsCoordinator.sol
function createUniqueStakeRewardsSubmission(...) {
   // ...
   // Checks if msg.sender is opted in
   rewardsSubmission.amount = _takeProtocolFee(msg.sender, rewardsSubmission.token,
rewardsSubmission.amount);
   // ...
}
```

This logic is flawed because when these functions are called by the EmissionsController during a reward distribution (pressButton). In this context:

1. The OperatorSet belongs to a specific AVS. This AVS may have explicitly opted in to pay protocol fees.
2. However, the caller (msg.sender) is the EmissionsController contract.

3. The EmissionsController contract doesn't call setOptInForProtocolFee and defaults to `false` in the isOptedInForProtocolFee mapping.

As a result, _takeProtocolFee sees that the EmissionsController is not opted in and skips the fee collection. While other functions like createOperatorDirectedOperatorSetRewardsSubmission and createOperatorDirectedAVSRewardsSubmission use the AVS address to determine whether the protocol should take a fee, the said functions (createUniqueStakeRewardsSubmission and createTotalStakeRewardsSubmission) use msg.sender as a source of truth. That means the protocol will lose protocol fees in case msg.sender is not opted in for fees and the AVS is.

Recommendations:

Modify createUniqueStakeRewardsSubmission and createTotalStakeRewardsSubmission to pass the AVS address to the fee helper.

Customer Response:

Fixed in commit 0015f920

"*The protocol fee mechanism should be based on who is actually calling the function (msg.sender), not a parameter that could represent different entities. This creates a consistent, predictable fee model where the caller's opt-in status determines fee application.*"

Fix Review:

Fixed

Low Severity Issues

L-01: Inconsistent reward amount validation against MAX_REWARDS_AMOUNT in RewardsCoordinator

Severity: Low	Impact: Low	Likelihood: Medium
Files: /src/contracts/core/RewardsCoordinator.sol#L678	Status: Fixed	

Description:

The RewardsCoordinator contract enforces a safety cap on rewards using the MAX_REWARDS_AMOUNT constant (1e38 - 1) to ensure compatibility with off-chain calculation logic. However, this check is applied inconsistently across submission types. In standard submissions validated by _validateRewardsSubmission, the check is performed on the raw amount provided in the input, before protocol fees are deducted. In contrast, _validateOperatorDirectedRewardsSubmission performs this check on the amountAfterFee. This discrepancy means that an operator-directed submission could technically provide a total amount slightly exceeding the cap as long as the net amount after the 20% protocol fee falls below it, whereas other submission types would revert in the same scenario.

Recommendations:

Standardize validation to check the total amount (before fees) against MAX_REWARDS_AMOUNT across all submission functions.

Customer Response:

Fixed in commit e26edc60

Fix Review:

Fixed

L-02: RewardsCoordinator.createAVSRewardsSubmission is vulnerable to event spamming and off-chain griefing

Severity: Low	Impact: Low	Likelihood: Medium
Files: src/contracts/core/RewardsCoordinator.sol#L476	Status: Acknowledged	

Description:

The createAVSRewardsSubmission function in RewardsCoordinator allows any caller to submit a rewards distribution. While the function correctly transfers the specified tokens from the caller (msg.sender is treated as the AVS), there are no restrictions on the token address or the minimum amount. A malicious actor can call this function using fake, worthless ERC20 tokens and 1-wei amounts to emit AVSRewardsSubmissionCreated events. This spamming behavior increases the storage load on the contract (via the isAVSRewardsSubmissionHash mapping) and, more importantly, griefs the off-chain nodes responsible for indexing these events and calculating global Merkle roots, as they must process every legitimate-looking submission.

Recommendations:

In case the function is meant to be permissionless, make sure to have proper validation on the nodes that process those events to deter spam.

Customer Response:

Acknowledged
"The permissionless nature of createAVSRewardsSubmission is a design trade-off that enables flexibility for AVSs to submit rewards without requiring prior registration or whitelisting. While the audit correctly identifies potential for event spam, we believe the risk is manageable:

Current Mitigation

- *Off-chain indexers implement robust filtering and validation*
- *Monitoring dashboards track submission patterns and flag anomalies "*

Fix Review:

Acknowledged

Informational Severity Issues

I-01: Use standard initializer for safer ownership assignment

Files:

src/contracts/core/EmissionsController.sol#L63

Description:

The initialize functions in EmissionsController and RewardsCoordinator set the contract owner by calling _transferOwnership directly. This internal function does not validate that the new owner is non-zero, creating a risk where a configuration error could inadvertently renounce ownership upon deployment.

Recommendations:

Use the standard OpenZeppelin __Ownable_init function instead of _transferOwnership. This aligns with best practices by including a built-in safety check that reverts if the initial owner is address(0).

Customer Response:

Acknowledged

"The direct use of _transferOwnership in the initialize functions is intentional and follows OpenZeppelin's upgradeability patterns used throughout the EigenLayer codebase. While __Ownable_init provides additional safety checks, our implementation includes:

- ***Deployment Script Validation:*** *All proxy deployments validate the initialOwner parameter before deployment*
- ***Consistent Pattern:*** *This pattern is used across multiple EigenLayer contracts (StrategyManager, AVSDirectory, etc.)*
- ***Gas Efficiency:*** *Direct _transferOwnership avoids unnecessary initialization steps*

The risk of passing address(0) is mitigated through deployment script checks rather than runtime checks. Changing this pattern would require updates across multiple contracts and is not warranted for this release."

Fix Review:

Acknowledged

I-02: Outdated documentation on pausable flags in RewardsCoordinatorStorage

Files:

src/contracts/core/storage/RewardsCoordinatorStorage.sol

Description:

The RewardsCoordinatorStorage contract defines several constants used as indices for the Pausable functionality. The NatSpec documentation (comments) above some of these constants references function names that do not exist in the current codebase.

The following inconsistencies were identified:

1. PAUSED_REWARD_ALL_STAKERS_AND_OPERATORS
   - *Comment says:* Pauses rewardAllStakersAndOperators
   - *Actual Function:* createRewardsForAllEarners
2. PAUSED_OPERATOR_DIRECTED_OPERATOR_SET_REWARDS_SUBMISSION
   - *Comment says:* Pauses setOperatorSetPerformanceRewardsSubmission
   - *Actual Function:* createOperatorDirectedOperatorSetRewardsSubmission
3. PAUSED_UNIQUE_STAKE_REWARDS_SUBMISSION
   - *Comment says:* Pauses createOperatorSetRewardsSubmission
   - *Actual Function:* createUniqueStakeRewardsSubmission

Recommendations:

Update the NatSpec comments in RewardsCoordinatorStorage.sol to accurately reflect the names of the functions that use these pause flags.

Customer Response:

Fixed in commit efcdc13e

Fix Review:

Fixed

I-03: EmissionsController._checkDistribution fails to validate operator sets for EigenDA distributions

Files:

src/contracts/core/EmissionsController.sol#L370

Description:

In EmissionsController.sol, the internal _checkDistribution function validates that the operatorSet provided in a distribution configuration is actually registered in the AllocationManager. Currently, this check is only applied to OperatorSetTotalStake and OperatorSetUniqueStake distribution types. However, the EigenDA distribution type also utilizes distribution.operatorSet.avs when calling createEigenDARewardsSubmission.

Recommendations:

Include DistributionType.EigenDA in the operator set registration check within _checkDistribution.

Customer Response:

Acknowledged

"The _checkDistribution function intentionally omits EigenDA from operator set validation because:

- ***Special Case Handling:*** *EigenDA distributions use a dedicated createEigenDARewardsSubmission pathway that handles the AVS reference differently than standard operator set distributions.*
- ***Validation Occurs Elsewhere:*** *The RewardsCoordinator.createEigenDARewardsSubmission function itself validates the AVS-level submission.*
- ***Configuration Control:*** *It's assumed Incentive Council (multisig) configures distributions and would not set an invalid EigenDA operator set as this would cause distribution failure during pressButton.*

  "

Fix Review:

Acknowledged

I-04: Outdated documentation for createRewardsForAllEarners in IRewardsCoordinator

Files:

src/contracts/interfaces/IRewardsCoordinator.sol#L490

Description:

The interface documentation (NatSpec) for the createRewardsForAllEarners function in IRewardsCoordinator.sol contains an outdated reference. It states that the function is 'only callable by the token hopper contract from the Eigen Foundation. However, with the introduction of the programmatic incentives system, this role is now primarily managed by the EmissionsController contract. Additionally, the code uses the onlyRewardsForAllSubmitter modifier, which is more general and allows any address authorized by the owner to call the function. The comment should be updated to reflect the actual governance and operational model.

Recommendations:

Update the NatSpec comment to correctly identify the EmissionsController and authorized submitters as the intended callers.

Customer Response:

Fixed in commit efcdc13e

Fix Review:

Fixed

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.